

Problemas de Decisão

- Problemas caracterizados por uma classe de perguntas, cujas respostas devem ser SIM ou NÃO:
 - Dado um DFA M e um string w , M aceita w ?
 - Dado um string w , w pode ser gerado por uma CFG?
 - Dado um polinômio P com coeficientes inteiros, P possui raízes inteiras?
 - Dada uma MT M e um string w , M pára com entrada w ?
- Dizemos que um problema de decisão é computável, ou decidível, se existe um algoritmo que o solucione.

Problemas de Decisão

- Podemos usar linguagens para representar problemas de decisão
- Exemplo - testar se um DFA M reconhece w :
 - $A_{\text{DFA}} = \{ \langle M, w \rangle \mid M \text{ é um DFA que aceita o string } w \}$
- Portanto, o problema é decidível se a linguagem que o representa é uma linguagem Turing-decidível.

Problemas Decidíveis

$A_{\text{DFA}} = \{ \langle D, w \rangle \mid D \text{ é um DFA que aceita o string } w \}$

- Teorema: A_{DFA} é decidível
- Prova: mostrar uma MT M_D que decide A_{DFA} .
 - $M_D =$ Entrada: $\langle D, w \rangle$ onde D é um DFA e w é um string
 1. Simula D com entrada w
 2. Se a simulação termina em um estado final, aceita; caso contrário, rejeita.

Problemas Decidíveis

$A_{\text{NFA}} = \{ \langle N, w \rangle \mid N \text{ é um NFA que aceita o string } w \}$

- Teorema: A_{NFA} é decidível
- Prova: mostrar uma MT M_N que decide A_{NFA} .
 - M_N = Entrada: $\langle N, w \rangle$ onde N é um NFA e w é um string
 1. Converte NFA N para DFA equivalente D
 2. Executa a MT M_D com entrada $\langle D, w \rangle$
 3. Se M_D aceita, então aceita; senão rejeita.

Problemas Decidíveis

$A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ é uma expressão regular que descreve uma linguagem que contém } w \}$

- Teorema: A_{REX} é decidível
- Prova: mostrar uma MT M_R que decide A_{REX} .
 - M_R = Entrada: $\langle R, w \rangle$ onde R é uma exp. regular e w é um string
 1. Converte a exp. regular R em um DFA equivalente D
 2. Executa a MT M_D com entrada $\langle D, w \rangle$
 3. Se M_D aceita, então aceita; senão rejeita

Problemas Decidíveis

$$E_{\text{DFA}} = \{ \langle D \rangle \mid D \text{ é um DFA e } L(D) = \emptyset \}$$

- Teorema: E_{DFA} é decidível
- Prova: mostrar uma MT M_{ED} que decide E_{DFA} .
 - M_{ED} = Entrada: $\langle D \rangle$ onde D é um DFA
 1. Marca o estado inicial do DFA D
 2. Repete, até que não haja mais nenhum estado a marcar:
Marca todo estado que tem uma transição
com origem em um estado já marcado
 3. Se nenhum estado de aceitação foi marcado, aceita;
caso contrário rejeita

Problemas Decidíveis

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ são DFAs e } L(A) = L(B) \}$$

- Teorema: EQ_{DFA} é decidível
- Prova: construir um DFA C tal que
 - $L(C) = (L(A) \cap L(B)') \cup (L(A)' \cap L(B))$
 - $L(A) = L(B)$ somente se $L(C) = \emptyset$
- M_{EQD} = Entrada: $\langle A, B \rangle$ onde A e B são DFAs
 1. Constrói o DFA C descrito acima
 2. Executa a MT M_{ED} com entrada C
 3. Se M_{ED} aceita, então aceita; caso contrário rejeita

Problemas Decidíveis

$$A_{CGF} = \{ \langle G, w \rangle \mid G \text{ é um GLC que gera o string } w \}$$

➤ Teorema: A_{CFG} é decidível

- Tentar gerar todas as possíveis derivações de G e testar se alguma deriva w não funciona
- Para obter uma MT M_G que decide A_{CGF} , devemos garantir que M_G testa apenas um número finito de derivações de G .
- Se G está na forma normal de Chomsky, qualquer derivação em G , de um string w tal que $|w| = n$, tem exatamente $2n - 1$ passos:
 - Portanto basta testar derivações com $2n - 1$ passos

Problemas Decidíveis

- M_G = Entrada: $\langle G, w \rangle$ onde G é uma CFG e w é um string
 1. Converte G para uma CFG equivalente G' , na forma normal de Chomsky
 2. Lista todas as derivações de G' com $2n - 1$ passos, onde n é o comprimento de w
 3. Se alguma dessas derivações gera w , aceita; caso contrário rejeita

Tese de Church-Turing para Problemas de Decisão:

- Existe um procedimento efetivo para solucionar um PD se, e somente se, existe uma MT que pare para todos os strings de entrada e que solucione o problema.
- Tese de Church-Turing estendida para PD's:
 - Um PD P é parcialmente solúvel se, e somente se, existe uma MT que aceita exatamente os elementos de P cuja resposta é sim.
- Sistemas equivalentes:
 - Funções recursivas (Kleene, 1936)
 - λ -calculus (Church, 1941)
 - Máquinas de registradores

Tese de Church-Turing para Problemas de Decisão:

- Todos esses modelos são equivalentes, isto é, definem a mesma classe de funções computáveis
- Note a natureza da asserção:
 - A tese de Church-Turing não é um teorema matemático – não pode ser provado, mas ...
 - Até hoje, não foi obtida nenhuma função intuitivamente computável que não seja computável por qualquer desses formalismos
- Essas evidências levaram à Tese de Church-Turing:
 - Uma função é computável se, e somente se, é computável por uma máquina de Turing.

Computabilidade

- Apesar do grande poder de computação de uma máquina de Turing - maior do que de qualquer computador real - existem problemas que não são Turing-computáveis
- De acordo com a Tese de Church-Turing, tais problemas não são solúveis por meio de um algoritmo
- Quais são os limites da computabilidade?